



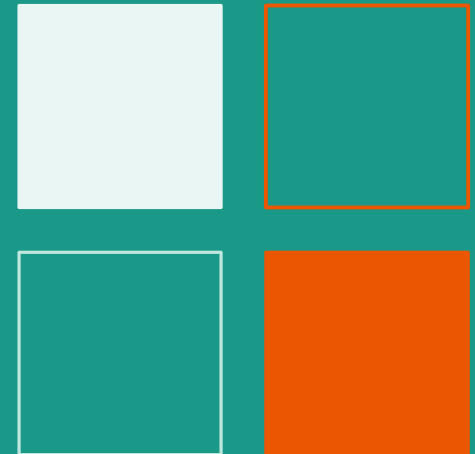
FORMATION

Apache Airflow 3

Orchestration de workflows de données

3 jours · 21 heures

Freelance Data Engineering



JOUR 1

Fondamentaux, installation & maîtrise des DAGs

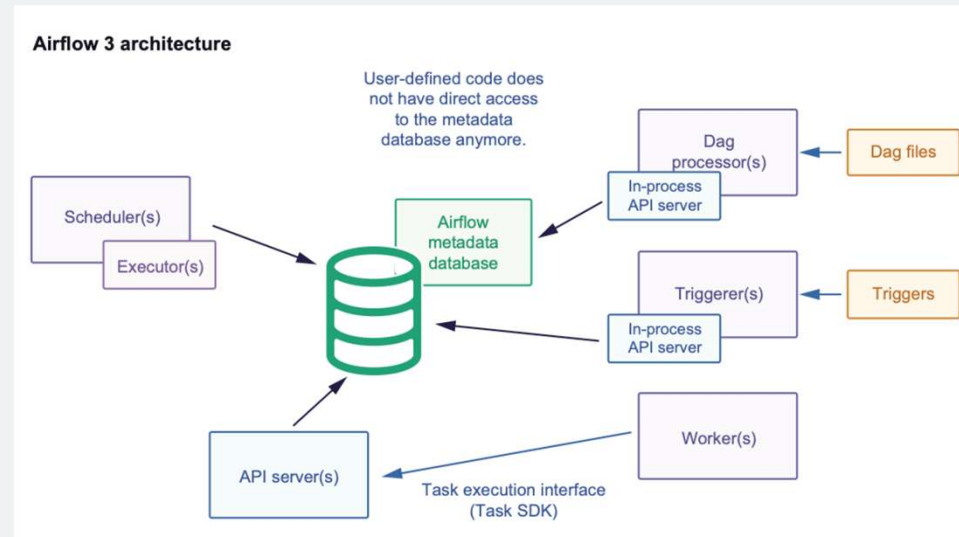


AU PROGRAMME

- Data orchestration & rôle d'Airflow
- Architecture des composants
- Installation : les 3 voies + Astro CLI
- Airflow UI, CLI et API REST v2
- Écrire un DAG : classique & TaskFlow
- Scheduling, catchup & backfill
- TP debug d'un DAG saboté

Formation Airflow

- Formation Apache Airflow — Orchestration de workflows de données (Airflow 3.3).
16/09/2026
- Formateur : Ancelin Lavaux





Objectifs

- Concevoir, déployer et maintenir des workflows robustes, observables et scalables avec **Airflow 3**
- Modéliser vos traitements en **DAGs clairs, maintenables et testables**
- Choisir et configurer les bons **executors** (du PoC local à la production)
- Mettre en œuvre les **nouveautés d'Airflow 3** : assets, data-aware & event-driven scheduling, DAG versioning, nouvelle UI



Présentation du formateur

01

Ancelin Lavaux

02

9 ans d'expérience en Data Engineering

03

Usage d'Airflow en production.



Tour de table

01

Qui êtes-vous ?

02

expérience Python / orchestration ?

03

venez-vous d'Airflow 2 ?

04

vos attentes ?



Déroulé & règle du jeu

- J1 : fondamentaux, installation, DAGs & scheduling · J2 : data-aware, données, organisation, tests, concepts avancés · J3 : nouveautés Airflow 3, intégration, executors, sécurité, projet
- Fil rouge : pipeline ETL **Snowystore** (e-commerce) construit brique par brique
- 70 % pratique : vous codez, je code avec vous · défis bonus pour les rapides

01



Introduction & fondamentaux Airflow

Rappels : la data orchestration

- Orchestrer = enchaîner des traitements dans le bon ordre, de façon fiable et répétée.
- Enjeux : dépendances, **reprise automatique** sur échec, **scaling**, observabilité. Cas type : ETL.



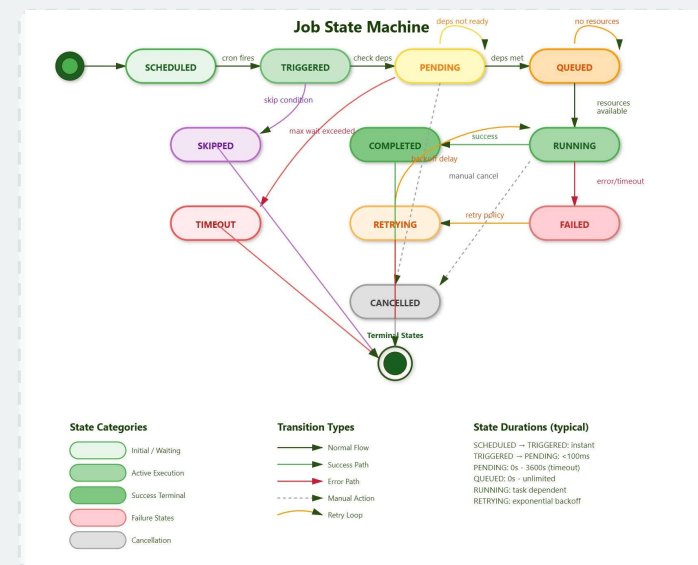
Qu'est-ce qu'Apache Airflow ?

- Plateforme open-source pour **créer, planifier et superviser des workflows par le code (Python)**. « Workflows as code ».
- Un **orchestrateur**, pas un moteur de traitement.



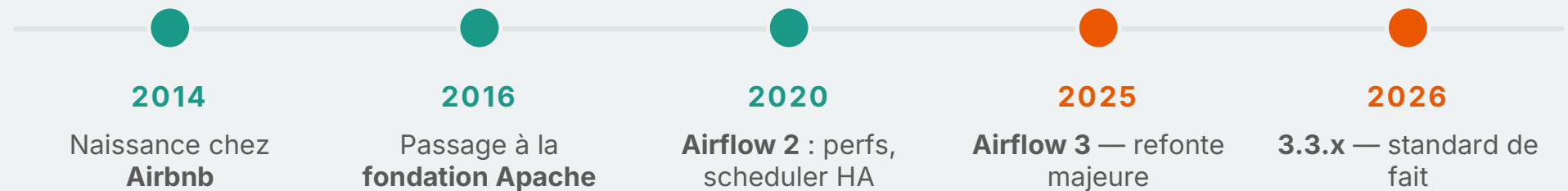
Le problème résolu & les avantages

- Avant : **cron** éparpillés, échecs silencieux, aucune visibilité ni dépendance.
- Airflow : dépendances explicites, retries, logs centralisés, monitoring, alerting.



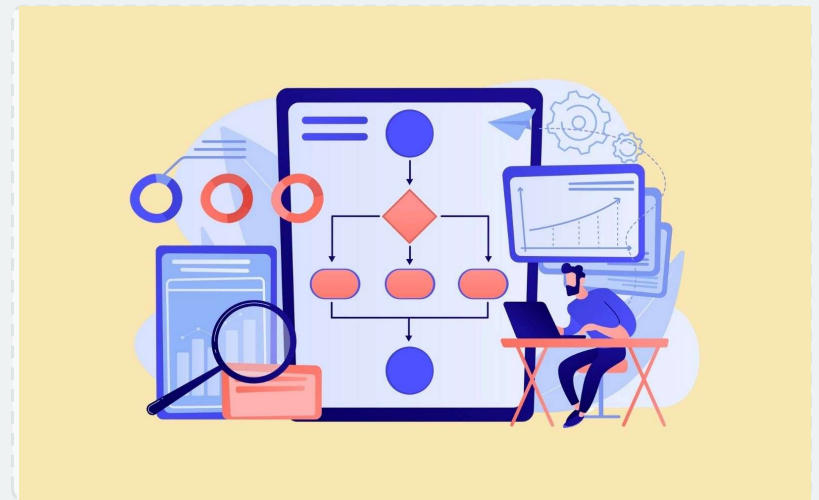


Historique & chronologie



Portabilité dans l'écosystème Big Data

- Airflow *pilote* sans se substituer : Spark, dbt, Snowflake, BigQuery, S3/GCS, Kafka, Kubernetes...
- Il orchestre, les outils exécutent (via les *providers*)





Pourquoi Airflow ?

01

Orchestration à grande échelle (des centaines de pipelines)

02

intégration avec les outils tiers (providers)

03

communauté & support massifs

04

maturité.



Airflow vs autres orchestrateurs

- **cron** (pas de dépendances/monitoring)
- **NiFi** (flux temps réel visuel)
- **Luigi** (ancêtre)
- **Prefect / Dagster** (concurrents Python modernes). Force d'Airflow : maturité + écosystème + communauté.



Cas d'usage courants

- ETL/ELT (majoritaire)
- alimentation de DWH/lakes
- ML
- reporting
- automatisation
- synchronisation de systèmes.



Le vocabulaire clé

- **DAG** (le workflow)
- **Task** (une étape)
- **Operator** (modèle de tâche)
- **DAG Run** (exécution datée)
- **Task Instance**. Acyclique = pas de boucle.

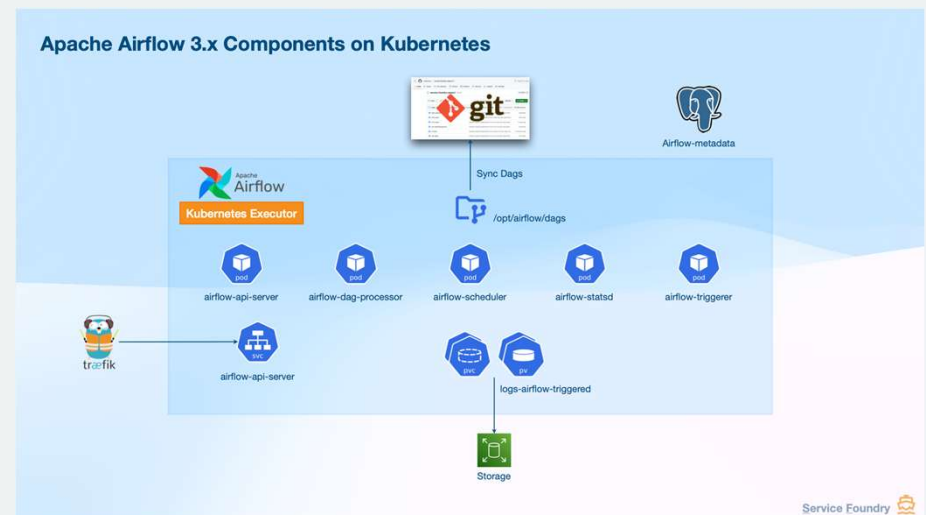


Architecture : vue d'ensemble

- Metadata DB au centre (source de vérité).
- Autour : DAG Processor, Scheduler, Executor/Workers, API Server (UI+API), Triggerer. Tout dialogue via la base.

Architecture : Scheduler & DAG Processor

- **Scheduler** : détermine les tâches prêtes et les confie à l'Executor (HA).
- **DAG Processor** : parse les fichiers, sérialise les DAGs — **séparé et obligatoire en Airflow 3.**





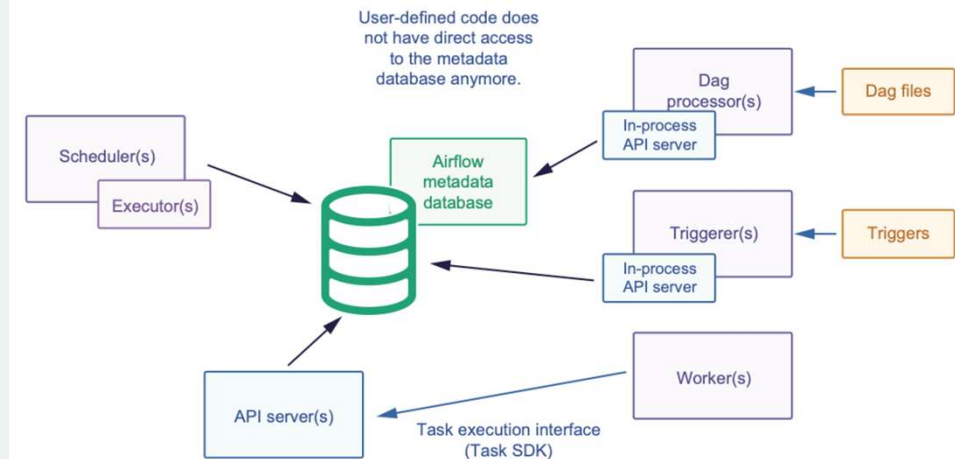
Architecture : API Server, UI & Metadata DB

- **API Server** (ex-Webserver) sert l'UI React + l'API REST.
- **Metadata DB** (Postgres) : tout l'état.
- **v3** : les tâches passent par l'API (Task SDK), plus d'accès direct à la DB.

Architecture : Executor, Workers & Triggerer

- **Executor** = stratégie d'exécution (Local/Celery/Kubernetes). **Workers** = exécutent.
- **Triggerer** = gère les tâches différées (deferrable) sans consommer de slot.

Airflow 3 architecture



02



Installation & prise en main



Les trois voies d'installation

01

1) **Classique** (pip + constraints)

02

2) **Docker** (docker-compose officiel)

03

3) **Astro CLI** (retenue ici).
Chacune a son usage.



Prérequis du poste

01

Podman — libre, *rootless*, sans contrainte de licence (contrairement à Docker Desktop, payant au-delà de 250 salariés)

02

8 Go de RAM minimum, 16 Go recommandés
· 10 Go de disque libre

03

Ports **8080**, **5432** et **5433** libres

04

Droits **administrateur** + accès aux registres d'images (voir dépannage)



Installer sur Windows

WINDOWS

```
# 1. Activer WSL 2 (PowerShell en ADMINISTRATEUR), puis REDEMARRER
wsl --install
wsl --set-default-version 2

# 2. L'Astro CLI installe et pre-configure Podman (depuis la v1.32)
winget install -e --id Astronomer.Astro

# 3. Rouvrir PowerShell (pour recharger le PATH), puis verifier
astro version
podman --version

# Si 'astro' n'est pas reconnu : ajouter son dossier a la variable PATH.
```



Installer sur macOS & Linux

MACOS / LINUX

```
# --- macOS : Homebrew installe Podman par défaut avec l'Astro CLI ---  
brew install astro  
podman machine init && podman machine start  
  
# --- Linux : Podman est natif, aucune machine virtuelle ---  
sudo apt install -y podman          # ou : dnf install podman  
systemctl --user enable --now podman.socket  # socket compatible Docker  
curl -sSL install.astronomer.io | sudo bash -s  
  
# Verification commune  
podman run --rm hello-world
```



Brancher l'Astro CLI sur Podman

ÉTAPE OUBLIÉE

```
# L'ETAPE QUE TOUT LE MONDE OUBLIE : sans elle, l'Astro CLI cherche Docker
astro config set -g container.binary podman
astro config get -g container.binary          # doit afficher : podman

# Windows / macOS : Podman tourne dans une machine virtuelle
podman machine start                         # si 'cannot connect to Podman'
podman machine set --memory 8192 --cpus 4    # si Airflow rame

# Linux rootless : en cas d'erreur de montage de volumes
astro config set -g duplicate_volumes false
```



Démarrer l'environnement Snowystore

DÉMARRAGE

```
cd snowystore-airflow

./scripts/check_env.sh          # poste, ports, RAM, acces au registre
python include/generate_orders.py --days 20    # donnees du fil rouge

astro dev start                  # http://localhost:8080 (admin/admin)

# Les commandes du quotidien
astro dev bash                  # shell DANS le conteneur
astro dev restart               # APRES modif Dockerfile / requirements
astro dev stop                  # arret propre
astro dev kill                  # reset complet
```



Dépannage express

	CAUSE	SOLUTION
cannot connect to Podman	VM Podman arrêtée	<code>podman machine start</code>
astro cherche Docker	réglage non appliqué	<code>astro config set -g container.binary podman</code>
port 8080 occupé	autre service local	<code>astro config set webserver.port 8081</code>
provider introuvable	image non reconstruite	<code>astro dev restart</code>
error pulling image	proxy d'entreprise	<code>ouvrir astrocrpublic.azurecr.io + docker.io</code>



Installation classique (pip + constraints)

01

```
pip install "apache-  
airflow==3.3.0" --  
constraint  
<constraints-3.12.txt> .  
airflow standalone
```

02

Toujours le fichier de
constraints (dépendances
lourdes)

03

*Détail des commandes en
annexe*



Installation via Docker (compose officiel)

01

`docker-compose.yaml`
officiel Airflow → tous les
composants conteneurisés

02

Proche de la production ;
verbeux à configurer

03

*On le manipule
concrètement au TP
CeleryExecutor (J3)*

Astro CLI : présentation & avantages

- Outil open-source d'Astronomer. Encapsule Docker Compose.
- **Avantages industrialisation** : projet standardisé, versions figées (Dockerfile), reproductibilité, proche du déploiement Astro Cloud.





Astro : création & anatomie d'un projet

- `astro dev init` → `dags/`, `include/`,
`plugins/`, `tests/`, `Dockerfile` (= version
Airflow), `requirements.txt`,
`airflow_settings.yaml`.



Astro : cycle de vie de l'environnement local

- `astro dev start` (UI :8080)
- `stop`
- `kill` (reset)
- `restart` (après Dockerfile/requirements)
- `logs -f`.



TP0 : Démarrer votre Airflow

PRATIQUE

20 MIN

-  Point de départ : le dossier du projet, celui qui contient le Dockerfile. Vérifier `podman --version` et `astro version`
-  Mission 1 : lancer `astro dev start`, puis ouvrir `http://localhost:8080` (admin / admin)
-  Mission 2 : Admin → Connections doit lister `snowstore_dwh`. « Aucun DAG trouvé » est NORMAL : dags/ est vide



TP0 : Anatomie du projet & checkpoint

CHECKPOINT

15 MIN

- ✓ Explorer `dags/`, `include/`, `Dockerfile`, `requirements.txt`, `airflow_settings.yaml` (connexion `snowystore_dwh`), `docker-compose.override.yml`
- ✓ UI verte · un DAG lancé · vous situez les Connections/Variables/Pools (Admin)



Airflow UI : tour d'horizon

PRATIQUE GUIDÉE

- 1 Nouvelle UI React (v3).
- 2 Navigation : liste des DAGs, Grid, Graph, Gantt, logs, code, **Assets**, Admin (Connections/Variables/Pools). Filtres, tags, pause.



Airflow CLI : commandes essentielles

01

```
airflow db migrate (init/maj schéma)
```

02

```
airflow db clean (purge historique)
```

03

```
airflow dags list / tasks list
```

04

```
airflow dags pause/unpause. Avec Astro :  
astro dev bash puis airflow ...
```



Airflow CLI : tester & rejouer

01

```
airflow tasks test  
<dag> <task> <date>  
(exécute une tâche  
isolément, sans planif)
```

02

```
airflow dags test <dag>  
<date>
```

03

```
airflow dags backfill  
(rejouer une plage). Outils  
de dev/debug.
```

TP0 bis : la CLI en main

PRATIQUE

- 1 Mission 1 : entrer dans le conteneur — `astro dev bash` (sous Git Bash : `winpty astro dev bash`)
- 2 Mission 2 : lancer `airflow version`, puis `airflow dags list`
- 3 Mission 3 : `airflow dags list-import-errors` — seul `snowystore_broken.py` apparaît (il est cassé exprès)
- + Défi bonus : sortir avec `exit`, puis lancer sans entrer : `astro dev bash -c "airflow version"`

Exercice : lire un run dans l'interface

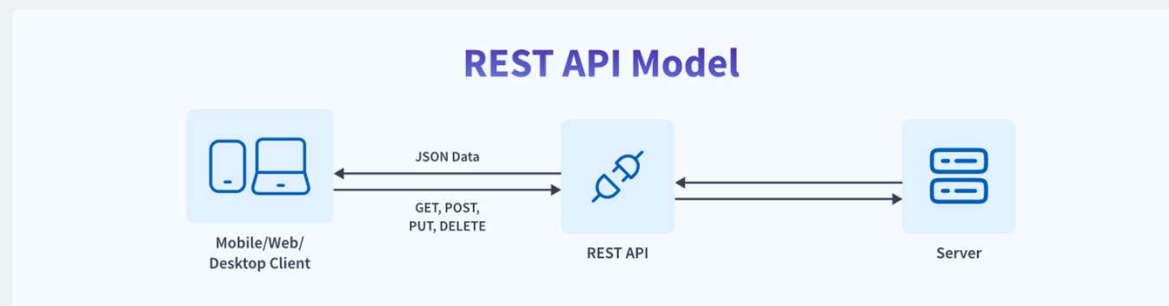
PRATIQUE

30 MIN

- + Point de départ : copier `solutions/demo_premier_run.py` dans `dags/` — c'est une démo, pas un exercice à écrire
- 1 Mission 1 : attendre ~30 s qu'il apparaisse, puis cliquer sur Déclencher SANS activer l'interrupteur. Constater que rien ne démarre
- 2 Mission 2 : ACTIVER l'interrupteur, redéclencher. Vue Grid : repérer les 4 tâches et leurs durées très différentes
- 3 Mission 3 : vue Graph — repérer la fourche en 2 branches parallèles, puis la jonction qui attend les deux
- 4 Mission 4 : ouvrir les Logs de publier pour y retrouver le rapport, puis l'onglet XCom pour voir les valeurs transmises

L'API REST v2 (Airflow 3)

- API REST stable (API Server) :
lister/déclencher des DAGs, lire l'état, gérer connexions/variables
- Documentation OpenAPI intégrée · pilotage externe





API REST : premiers appels

PRATIQUE GUIDÉE

- 1 **Ensemble** : `GET /api/v2/dags` (lister) · `POST .../dagRuns` (déclencher) · `GET .../dagRuns/<id>` (état)
- 2 Auth par token · exemple `curl`
- 3 Vous refaites chaque appel en même temps que moi



Questions & reprise — fin de matinée J1

01

Vos questions sur :
fondamentaux,
architecture, installation,
UI/CLI/API

02

Reprise des points signalés
comme flous

03



Maîtriser les DAGs



Concepts de base

- Rappel : **DAG** (workflow) → **Tasks** (étapes) → reliées par des **dépendances** → chaque tâche = un **Operator**.

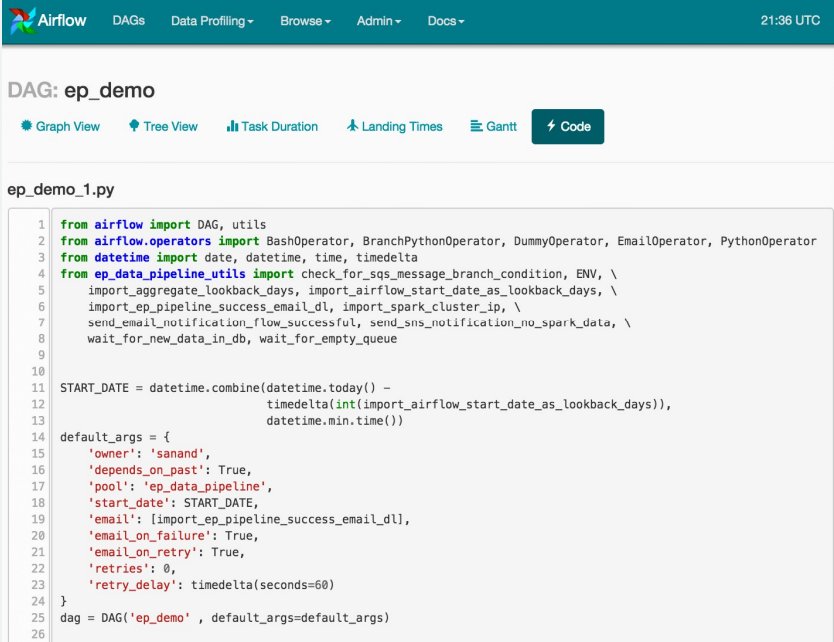
Anatomie d'un DAG

- Un `.py` dans `dags/` : imports → définition (id, schedule, start_date) → tâches → dépendances. Détecté automatiquement.

```
hello_world_dag.py
1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from datetime import datetime
4
5 def helloWorld():
6     print('Hello World')
7
8 with DAG(dag_id="hello_world_dag",
9         start_date=datetime(2021,1,1),
10        schedule_interval="@hourly",
11        catchup=False) as dag:
12
13     task1 = PythonOperator(
14         task_id="hello_world",
15         python_callable=helloWorld)
16
17 task1
```

Approche « classique » (with DAG)

- with `DAG(dag_id=..., schedule=..., start_date=...)` as `dag`: puis instantiation explicite des Operators. Explicite, verbeux.



The screenshot shows the Apache Airflow web interface. At the top, there's a navigation bar with links for DAGs, Data Profiling, Browse, Admin, and Docs. The current view is for a DAG named 'ep_demo'. Below the navigation bar, there are tabs for Graph View, Tree View, Task Duration, Landing Times, and Gantt. A 'Code' button is also present. The main content area displays the Python code for the DAG, which is 'ep_demo_1.py'. The code includes imports for DAG and various operators, defines a START_DATE, and creates a DAG object with default arguments.

```
1 from airflow import DAG, utils
2 from airflow.operators import BashOperator, BranchPythonOperator, DummyOperator, EmailOperator, PythonOperator
3 from datetime import date, datetime, time, timedelta
4 from ep_data_pipeline_utils import check_for_sqs_message_branch_condition, ENV, \
5     import_aggregate_lookback_days, import_airflow_start_date_as_lookback_days, \
6     import_ep_pipeline_success_email_dl, import_spark_cluster_ip, \
7     send_email_notification_flow_successful, send_sns_notification_no_spark_data, \
8     wait_for_new_data_in_db, wait_for_empty_queue
9
10
11 START_DATE = datetime.combine(datetime.today() -
12                               timedelta(int(import_airflow_start_date_as_lookback_days)),
13                               datetime.min.time())
14
15 default_args = {
16     'owner': 'sanand',
17     'depends_on_past': True,
18     'pool': 'ep_data_pipeline',
19     'start_date': START_DATE,
20     'email': [import_ep_pipeline_success_email_dl],
21     'email_on_failure': True,
22     'email_on_retry': True,
23     'retries': 0,
24     'retry_delay': timedelta(seconds=60)
25 }
26 dag = DAG('ep_demo', default_args=default_args)
```



Approche décorateurs (@dag, @task — TaskFlow)

01

Décorateurs `@dag` et `@task` sur des fonctions Python.

02

XComs et dépendances **automatiques**.

03

Style recommandé au quotidien.



Les Operators : la brique de base

01

Operator = modèle de tâche.

02

Familles : **action**
(Bash/Python), **transfert**,
sensor.

03

Des centaines via les *providers*.



BashOperator

- Exécute une commande/script shell.
`bash_command`, `env`, `cwd`.
- Renvoie la dernière ligne stdout en XCom.
Jinja sur `bash_command`.

```
1 from airflow.operators.bash_operator import BashOperator
2
3 run_db_query = BashOperator(
4     task_id='run_db_query',
5     bash_command='psql -c "SELECT * FROM table_name"',
6     dag=dag
7 )
8
```

PythonOperator

- Exécute une fonction (`python_callable`), `op_args/op_kwargs`, accès au **contexte** (`**context`, dates du run).

```
airflow_operators.py > ...
1  from airflow import DAG
2  from airflow.operators.python import PythonOperator
3  from datetime import datetime
4
5
6  def print_hello():
7      return "Hello from Anubhav!"
8
9
10 with DAG(
11     "python_operator_example",
12     start_date=datetime(2024, 9, 15),
13     schedule_interval="@daily",
14 ) as dag:
15     hello_task = PythonOperator(task_id="hello_task", python_callable=print_hello)
16
```



EmptyOperator (+ encart renommage)

AIRFLOW 2 → 3

- Tâche qui ne fait rien : jonction/repère (start, end, join).
- **2.x → 3.x** : `DummyOperator` → `EmptyOperator`.



Focus : les paramètres d'un DAG

- `dag_id`
- `schedule`
- `start_date`
- `catchup`
- `tags`
- `default_args` (owner, retries, retry_delay)

Définir les dépendances

- `>>` (downstream), `<<` (upstream), `a >> [b, c]` (parallèle), helpers `chain()`, `cross_downstream()`.

The screenshot displays the Apache Airflow web interface. On the left, a sidebar contains navigation links for Home, DAGs, Assets, Browse, and Admin. The main area shows a DAG named 'demo' with a status of 'Success' and a run time of '2025-09-18 16:40:33'. The DAG is visualized as a graph with two tasks: 'hello' (BashOperator) and 'airflow' (@task). Both tasks are marked as 'success'. The right panel shows the DAG's code, which defines the tasks and their dependencies using the 'airflow' task as a dependency for 'hello'.

```
1 from datetime import datetime
2
3 from airflow.sdk import DAG, task
4 from airflow.providers.standard.operators.bash import BashOperator
5
6 # A Dag represents a workflow, a collection of tasks
7 with DAG(dag_id="demo", start_date=datetime(2022, 1, 1), schedule="@ * * * *" as dag:
8     # Tasks are represented as operators
9     hello = BashOperator(task_id="hello", bash_command="echo hello")
10
11
12 @task()
13 def airflow():
14     print("airflow")
15
16
17 # Set dependencies between tasks
18 hello >> airflow()
19
```

Micro-exercice : câbler les dépendances

PRATIQUE

10 MIN

- 1 Sur un DAG jetable : câbler `a >> [b, c] >> d` (fan-out puis fan-in)
- 2 Ajouter `e` qui ne part qu'après `d` ; vérifier la forme dans la vue **Graph**
- + **Bonus** : utiliser `chain()` pour réécrire la même chaîne

Un DAG en pause ne s'exécute pas

PIÈGE

SYMPTÔME

Le run se crée, les tâches s'affichent... et restent **indéfiniment en file** (*queued*).
Aucun message d'erreur.

CAUSE

Le DAG est **en pause** — c'est l'état **par défaut** de tout nouveau DAG. Le scheduler ne programme alors **aucune** de ses tâches, *même si vous déclenchez un run à la main.*

SOLUTION

Activer l'interrupteur à gauche du nom du DAG. Ou, dans la fenêtre de déclenchement, cocher « **Réactiver ... lors du déclenchement** ».

*Pourquoi cette pause par défaut ? C'est une **protection** : on déploie un DAG, on le relit, et on ne l'active que lorsqu'on est sûr de lui. Imaginez un DAG mal réglé qui part seul en production, la nuit de son déploiement.*

TP1 : Votre premier DAG Snowystore

PRATIQUE

35 MIN

- 1 Mission 1 : créer dags/snowystore_etl.py — DAG snowystore_etl, schedule="@daily", catchup=False, tag "snowystore", default_args avec retries=2
- 2 Mission 2 : dans CE MÊME FICHIER, 3 tâches — extract (BashOperator), transform (PythonOperator qui logge), load (EmptyOperator)
- 3 Mission 3 : chaîner extract >> transform >> load, activer le DAG, le déclencher, lire les logs de chaque tâche

Défi bonus : réécrire le même DAG en TaskFlow (@dag / @task) dans un second fichier



TP1 : Restitution

CHECKPOINT

10 MIN

- ✓ DAG vert
- ✓ 3 tâches dans l'ordre
- ✓ logs accessibles

TP1 bis : Réécriture en TaskFlow

PRATIQUE

25 MIN

- + Point de départ : votre dags/snowystore_etl.py du TP1 (version classique)
- 1 Mission 1 : créer dags/snowystore_etl_taskflow.py — mêmes 3 tâches, avec les décorateurs @dag et @task
- 2 Mission 2 : supprimer les >> — passer le résultat d'une tâche à la suivante : load(transform(extract()))
- 3 Mission 3 : APPELER la fonction décorée en fin de fichier, sinon le DAG n'apparaît pas. Les 2 vues Graph doivent être identiques

Corrigé — TP1 : le DAG en TaskFlow

CORRIGÉ

```
@dag(dag_id="snowystore_etl", start_date=datetime(2026, 6, 1),
     schedule="@daily", catchup=False, tags=["snowystore"],
     default_args={"retries": 2, "retry_delay": timedelta(minutes=1)})
def snowystore_etl():

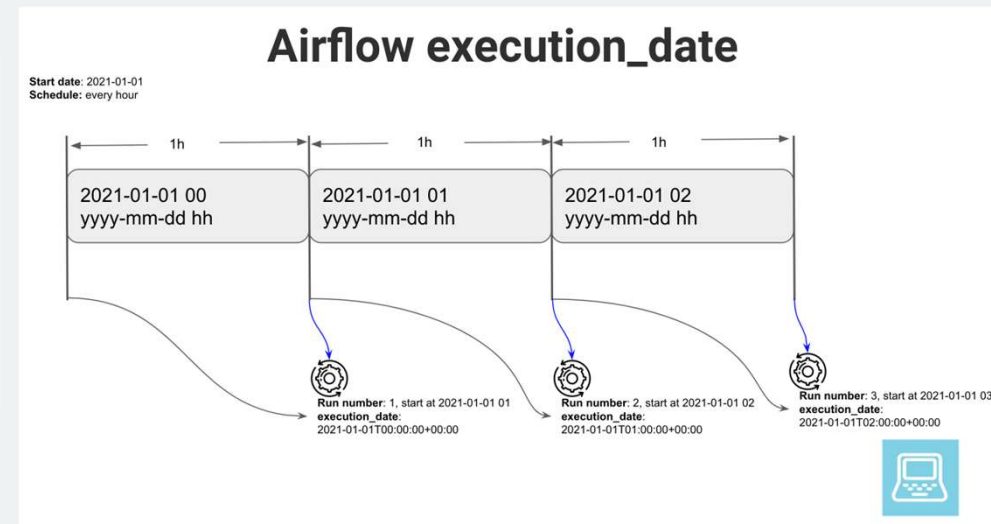
    @task
    def extract() -> int:
        return 1234                # le return devient un XCom

    @task
    def transform(n_rows: int) -> int:
        return n_rows

    # La dependance est deduite de l'appel : aucun >> a ecrire
    load(transform(extract()))
```

Scheduling : start_date, logical_date et ds

- start_date = depuis quand le DAG existe dans le temps.
- Airflow 3 (cron) : logical_date = data_interval_start = end = ds. Un point, pas un intervalle.





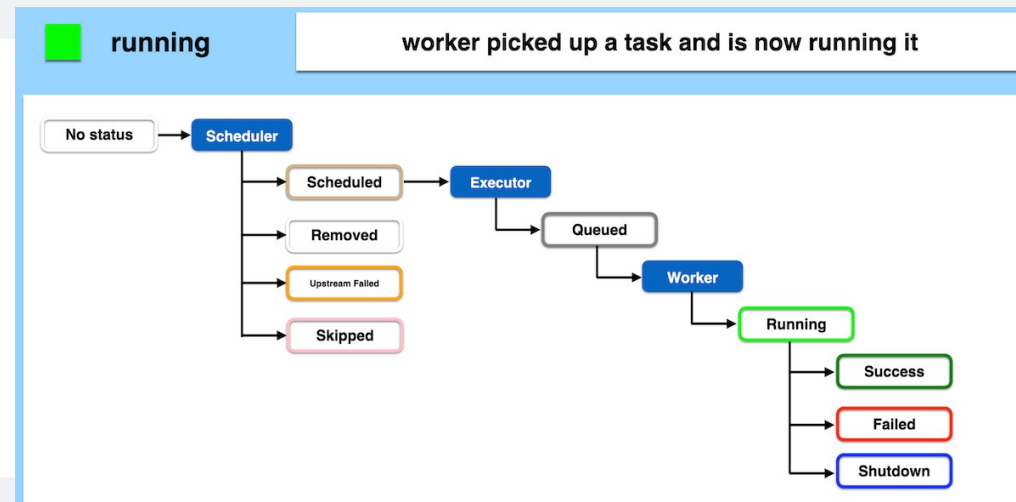
Scheduling : cron, presets & timedelta

AIRFLOW 2 → 3

- `schedule="0 6 * * *" (cron)`
- `presets @daily/@hourly`
- `None` (manuel)
- `timedelta(hours=6)` (glissant). **v3** : `schedule_interval` → `schedule`.

États des tâches (cycle de vie)

- none → scheduled → queued → running → success / failed
- Intermédiaires : `up_for_retry`, `upstream_failed`, `skipped`. Codes couleur dans la Grid





Catchup & backfill (+ idempotence)

01

`catchup=True` rattrape
tous les intervalles
manqués (des centaines
de runs)

02

`catchup=False`
(recommandé)

03

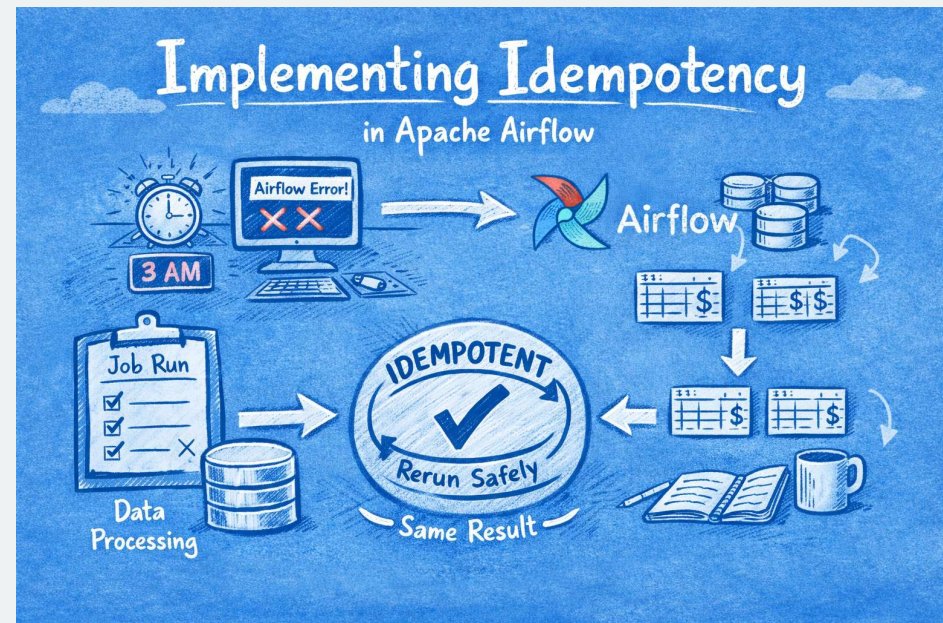
backfill = rejeu volontaire
d'une plage. Prerequisite :
idempotence.

Backfill via CLI et via UI (Airflow 3)

- CLI : `airflow dags backfill -s <start> -e <end> <dag>`.
- **v3** : backfill aussi **déclenchable et suivi depuis l'UI** (nouveau).
- Réglage `rerun_with_latest_version` (bundle).

Bonnes pratiques & anti-patterns

- Le DAG Processor **parse le fichier en boucle**
→ top-level léger, logique **dans les tâches**.
- Anti-patterns : `start_date = datetime.now()`, `catchup=True` non maîtrisé, tâches non idempotentes.



TP2 : Scheduling (unpause, catchup, backfill)

PRATIQUE

35 MIN

- + Point de départ : votre dags/snowystore_etl.py du TP1
- 1 Mission 1 : start_date FIXE 10 jours en arrière, schedule="@daily", catchup=False. Activer et observer quelle date porte le run
- 2 Mission 2 : dans une tâche, afficher logical_date, data_interval_start, data_interval_end et ds. Les comparer
- 3 Mission 3 : passer catchup=True, observer le rattrapage, REMETTRE catchup=False. Bonus : airflow dags backfill -s <début> -e <fin> snowystore_etl



TP2 : Restitution

CHECKPOINT

10 MIN

- ✓ vous savez dire quelle date porte le run, et pourquoi
- ✓ catchup observé puis désactivé
- ✓ backfill réussi.

Corrigé — TP2 : scheduling

CORRIGÉ

```
@dag(
    start_date=datetime(2026, 7, 20),    # date FIXE, jamais datetime.now()
    schedule="@daily",                  # v3 : "schedule", plus "schedule_interval"
    catchup=False,                      # True = rattrapage massif
)
def snowystore_scheduling():

    @task
    def show_interval(**context):
        print(context["data_interval_start"])    # debut de la periode traitee
        print(context["data_interval_end"])      # fin de la periode
        print(context["ds"])                    # date du run (souvent hier)

# Rejouer une plage : airflow dags backfill -s 2026-07-20 -e 2026-07-22 <dag>
```

TP2 bis : Debug d'un DAG saboté

PRATIQUE

35 MIN

- + Point de départ : dags/snowystore_broken.py, déjà présent et VOLONTAIREMENT cassé (ne pas ouvrir son corrigé)
- 1 Mission 1 : comprendre pourquoi il n'apparaît pas — airflow dags list-import-errors dans le conteneur
- 2 Mission 2 : localiser les 5 bugs avec le message d'erreur, la Grid et les logs uniquement
- 3 Mission 3 : corriger, puis relancer avec le bouton Clear jusqu'à un run 100 % vert
- + Défi bonus : écrire un test pytest qui aurait détecté ces bugs avant la production
- + Restitution : citer chaque bug et sa correction ; le plus sournois = la start_date dynamique

Corrigé — TP debug : les 5 bugs

CORRIGÉ

```
# 1. DummyOperator          -> EmptyOperator      (renomme en Airflow 3)
# 2. context['date_du_jour'] -> context['ds']       (cle inexistante)
# 3. start_date=datetime.now() -> date FIXE
#    LE PLUS SOURNOIS : aucune erreur, mais le DAG ne part JAMAIS.
#    Le DAG Processor relit le fichier en boucle : la date bouge a chaque
#    relecture, donc l'intervalle n'est jamais clos.
# 4. catchup=True           -> catchup=False       (rafale de rattrapage)
# 5. load >> extract        -> supprime           (un DAG est ACYCLIQUE)

with DAG(dag_id="snowystore_fixed",
        start_date=datetime(2026, 7, 20),
        schedule="@daily", catchup=False) as dag:
    extract >> check >> load          # plus de cycle
```

Récap Jour 1 / Quiz flash

- 5 questions : DAG/Task/Operator ? rôle du Scheduler & du DAG Processor ? 3 voies d'installation ? pourquoi catchup=False ? que vaut ds pour un run quotidien ?





Questions & reprise — fin de journée J1

01

Vos questions sur : DAGs,
operators, dépendances,
scheduling,
catchup/backfill

02

Reprise des points signalés
comme flous